

ECO5050 AI Applications and Techniques in Economics

Lecture 8 Ensemble Methods

Yaein Baek

Sogang University

Fall 2025

Outline

- 1 Forecast Combination
- 2 Ensemble Methods
- 3 Adaptive Boosting (AdaBoost)
- 4 Gradient Boosting

Ensembling and Model Averaging

- “Ensembling” is the term used in Machine Learning (ML) for model averaging across ML algorithms
- In the economic forecasting literature, this is often referred to as a “forecast combination”
 - ▶ If the null that two models have equal predictive accuracy is not rejected, do we use the forecast from the first model, the second model or both?
 - ▶ Even if one model seems to be better than the other, it is not clear that it is optimal to ignore the forecasts from the weaker model.
- Empirically, combining forecasts from multiple models often outperform forecasts from a single model.
 - ▶ Clemen (1989), Makridakis and Hibon (2000), Stock and Watson (1998), Marcellino (2004)

Forecast Combination

- An argument often used to justify forecast combinations is that they diversify against model uncertainty.
 - ▶ Individual forecasting models are likely to be misspecified: models are best viewed as simple approximations to a more complicated, constantly evolving reality.
 - ▶ To some extent forecast combination provides insurances against “breaks” or nonstationarities that may occur in the future.
- How much weight to assign to individual forecasts?
 - ▶ Bates and Granger (1969) suggested to simply regress y on the individual forecasts $f_i(z_t, \theta)$ with a constant. If f_i are unbiased, omit the intercept term and restrict the slope coefficients to sum to one $\rightarrow$ optimal combination weights under MSE loss
 - ▶ The optimality of these forecast weights holds for much broader class of loss functions (Elliott and Timmermann 2004)

Forecast Combination

- Empirical studies often find that simple equal-weighted forecast combinations outperform more sophisticated combination schemes that rely on estimated weights → Forecast combination puzzle
- The least squares combination weights of Bates and Granger (1969) are optimal in population, but estimation error from constructing the weights from data mean that the optimal loss is not achieved in finite sample.
 - ▶ Estimation error can overwhelm any gains from setting the weights to their optimal values rather than using equal weights.
- If the models use roughly the same data sources and empirical techniques so differences in the performance across models are small, giving each forecast identical weights can be relatively efficient.
- Bayesian Model Averaging (BMA): Raftery et al. (1997), Fernandez et al. (2001), Magnus et al. (2010)

Ensemble Methods

- Ensemble methods combine the predictions of a group of prediction models in order to improve performance/robustness over a single prediction model
 - ▶ A single prediction model is referred to as a base learner (e.g. Regression Tree)
 - ▶ A group of base learners is called an ensemble
- Ensemble methods include
 - ▶ Bagging (Bootstrap Aggregating)
 - ▶ Boosting
 - ▶ Voting classifiers
 - ▶ Stacking

Boosting

- In boosting, the ensemble consists of very simple base learners, also often referred to as weak learners.
- The general idea of most boosting methods is to train base learners *sequentially*, each trying to correct its predecessor.
- The most popular method is **AdaBoost** (adaptive boosting) and **Gradient Boosting**

AdaBoost Classification

- Consider the classification prediction where y is a finite discrete variable and the training set is $\{(y_i, X_i)\}_{i=1}^n$
- The ensemble consists of M base learners, so that the base learner is trained iteratively up to M rounds.
- Specify the learning rate hyperparameter $\eta > 0$.
 - ▶ Weight applied to the base learner at each boosting iteration (defaults to 1)
- Initialize sample weights: for the first iteration, all samples in the training set are assigned equal weight

$$w_i^{(1)} = \frac{1}{n}, \quad i = 1, \dots, n$$

AdaBoost Classification

For each iteration $m = 1, \dots, M$

- 1 Train the m th weak learner with $\{(y_i, X_i)\}_{i=1}^n$ and sample weights $\{w_i^{(m)}\}_{i=1}^n$
- 2 The weighted error rate of the m th weak learner

$$r_m = \frac{\sum_{i=1}^n w_i^{(m)} \mathbf{1}\{\hat{y}_i^{(m)} \neq y_i\}}{\sum_{i=1}^n w_i^{(m)}}$$

- 3 The m th weak learner's weight α_m determines the contribution to the final prediction

$$\alpha_m = \eta \log \left(\frac{1 - r_m}{r_m} \right).$$

The log term ensures that learners with low r_m receive higher weights.

AdaBoost Classification

- 4 The sample weights are updated for each training sample so that the incorrectly predicted samples have their weights increased

$$w_i^{(m+1)} = w_i^{(m)} \exp \left(\alpha_m \mathbf{1} \{ \hat{y}_i^{(m)} \neq y_i \} \right)$$

Normalize the weights so that $\sum_{i=1}^n w_i^{(m+1)} = 1$

- 5 Repeat steps 1-4 for each weak learner until M learners are trained or until improvement stops.
- 6 Compute the predictions of all weak learners $\hat{y}^{(m)}(x)$ and weigh them using the predictor weights α_m . The predicted class is the one that receives the majority of weighted votes.

$$\hat{y}(x) = \arg \max_k \sum_{m=1}^M \alpha_m \mathbf{1} \{ \hat{y}^{(m)}(x) = k \}$$

AdaBoost Classification

AdaBoostClassifier() [▶ Document](#)

- Example: AdaBoost classifier based on 30 decision stumps, learning rate $\eta = 0.5$

```
from sklearn.ensemble import AdaBoostClassifier

ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(max_depth=1), n_estimators=30,
    learning_rate=0.5, random_state=42)
ada_clf.fit(X_train, y_train)
```

AdaBoost Regression

- The same process when y is a continuous variable, except that the computation of the error rate and weights are different.
- Step 2: The weighted error rate

$$r_m = \frac{\sum_{i=1}^n w_i^{(m)} (y_i - h_m(X_i))^2}{\sum_{i=1}^n w_i^{(m)}}$$

- Step 4: Update the sample weights

$$w_i^{(m+1)} = w_i^{(m)} \exp(\alpha_m |y_i - h_m(X_i)|)$$

- ▶ If `AdaBoostRegressor(loss='linear')`, then $(1 + \alpha_m |y_i - h_m(X_i)|)$ instead of the exponential function.
- Step 6: The final AdaBoost prediction is the weighted average of all weak learner predictions $h_m(x)$ using weights α_m

$$\hat{y}(x) = \sum_{m=1}^M \alpha_m h_m(x)$$

AdaBoost Regression

AdaBoostRegressor() [▶ Document](#)

- Example: AdaBoost regression based on 100 trees, each with maximum depth 3, and learning rate $\eta = 0.5$

```
from sklearn.ensemble import AdaBoostRegressor

ada_reg = AdaBoostRegressor(DecisionTreeRegressor(max_depth=3),
                             n_estimators=100, learning_rate=0.5,
                             loss='exponential', random_state=42)
ada_reg.fit(X_train, y_train)
y_pred = ada_reg.predict(X_test)
```

Gradient Boosting

- Gradient Boosting fits a weak learner to the residuals made by the previous weak learner.
 - ▶ It can also be used for regression and classification.
- Consider the regression prediction where y is a continuous variable and the training set is $\{(y_i, X_i)\}_{i=1}^n$
- The ensemble consists of M weak learners (e.g. trees), so that the weak learner is trained iteratively up to M rounds.
- Specify the learning rate hyperparameter $\eta > 0$.
- Initialize the model with a baseline prediction

$$\hat{y}^{(0)} = \frac{1}{n} \sum_{i=1}^n y_i$$

Gradient Boosting

For each iteration $m = 1, \dots, M$

- 1 Compute the residuals $r_i^{(m)} = y_i - \hat{y}_i^{(m-1)}$
- 2 Train a base learner $h_m(x)$ on $\{(r_i^{(m)}, X_i)\}_{i=1}^n$
- 3 Update the model prediction by adding the m th base learner's prediction scaled by the learning rate

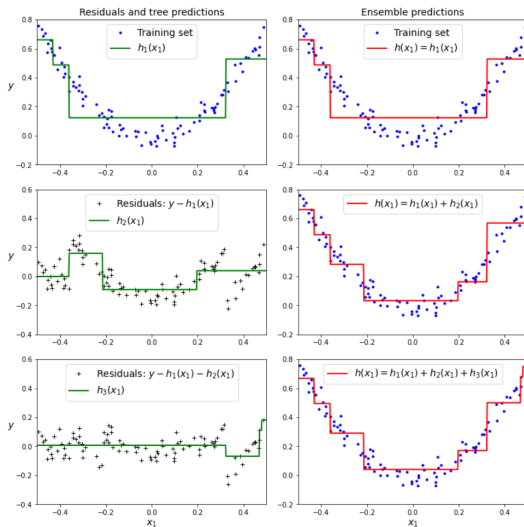
$$\hat{y}_i^{(m)} = \hat{y}_i^{(m-1)} + \eta \cdot h_m(X_i).$$

The learning rate controls how much the m th base learner's predictions (adjustments) contribute to the overall model.

- 4 The final Gradient Boosting prediction on a test sample x is

$$\hat{y}(x) = \hat{y}^{(0)} + \sum_{m=1}^M \eta \cdot h_m(x)$$

Gradient Boosting



Géron (2023) Figure 7-9

Gradient Boosting

GradientBoostingRegressor() [▶ Document](#)

- Gradient Boosted Regression Trees (GBRT): similar to the RandomForestRegressor class, it has hyperparameters to control the growth of trees (e.g. max_depth, min_samples_leaf) and hyperparameters to control the ensemble training, such as n_estimators.
- For classification use GradientBoostingClassifier()
- Example: Gradient boosting with 3 trees, each with maximum depth 2, and learning rate $\eta = 2$

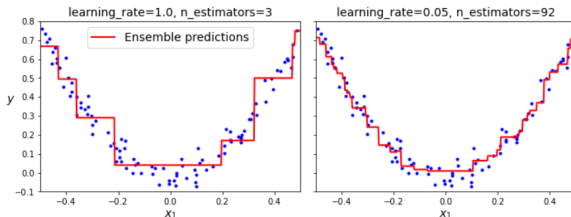
```
from sklearn.ensemble import GradientBoostingRegressor

gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=3,
                                 learning_rate=1.0, random_state=42)
gbrt.fit(X, y)
```

Gradient Boosting

`GradientBoostingRegressor()` [▶ Document](#)

- If the `learning_rate` is set to a low value, such as 0.05, you will need more trees in the ensemble to fit the training set but the predictions will usually generalize better (shrinkage)



Géron (2023) Figure 7-10

- However the GBRT will overfit the training set if there are too many trees

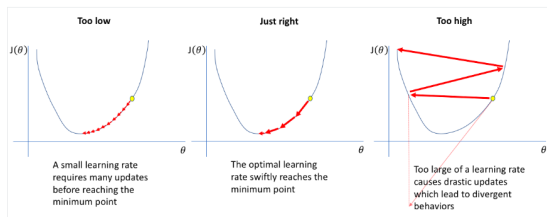
Gradient Boosting

`GradientBoostingRegressor()` [▶ Document](#)

- `n_iter_no_change`: If set to an integer value such as 10, then `GradientBoostingRegressor` will automatically stop adding more trees during training if it sees that the last 10 didn't help.
 - ▶ The `fit()` method automatically splits the training set into a smaller training set and validation set to evaluate the model's performance each time it adds a new tree; the size of the validation set is controlled by the `validation_fraction` hyperparameter
- `subsample`: specifies the fraction of training samples to be used for training each tree. `default=1.0`.
 - ▶ If smaller than 1.0 this results in Stochastic Gradient Boosting. It uses a random subset of the training data for training each base learner.

Gradient Descent

- **Gradient descent** is a generic optimization algorithm capable of finding optimal solutions to a wide range of problems
- It measures the local gradient of the cost function (e.g. MSE) with regard to the parameter vector θ , and it goes in the direction of the descending gradient.
 - ▶ In practice the initial θ are random values, then it is improved gradually each step at a time, attempting to decrease the cost function until the algorithm converges to the minimum.
- The size of the steps in gradient descent is determined by the learning rate



Source: [▶ Link](#)

Gradient Descent

- Although the MSE for a linear regression model is a convex function, not all cost functions are convex
- The gradient vector of the MSE is the vector of partial derivatives of the MSE for each model parameter θ_j

$$\nabla_{\theta} \text{MSE}(\theta) = \begin{bmatrix} \frac{\partial}{\partial \theta_1} \text{MSE}(\theta) \\ \frac{\partial}{\partial \theta_2} \text{MSE}(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_K} \text{MSE}(\theta) \end{bmatrix}$$

which is equal to $\frac{2}{n} \mathbf{X}'(\mathbf{X}\theta - \mathbf{Y})$ for a linear regression model.

- We move to the opposite direction of the gradient vector (i.e. negative gradient) multiplied by the learning rate η . The gradient descent step from m to $m + 1$ is

$$\theta^{(m+1)} = \theta^{(m)} - \eta \cdot \nabla_{\theta} \text{MSE}(\theta^{(m)})$$

Gradient Descent

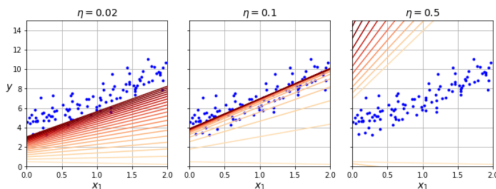
- The **Batch Gradient Descent** uses the whole batch of training data (the full training set) at every step
- Example: The batch gradient descent of a linear regression model

```
eta = 0.1 # learning rate
n_epochs = 1000
m = len(X_b) # number of instances

np.random.seed(42)
theta = np.random.randn(2, 1) # randomly initialized model parameters

for epoch in range(n_epochs):
    gradients = 2 / m * X_b.T @ (X_b @ theta - y)
    theta = theta - eta * gradients
```

The first 20 steps of gradient descent using three different learning rates



Géron (2023)

Gradient Descent

- The **Stochastic Gradient Descent (SGD)** picks a random sample in the training set at every step and computes the gradients based only on that single observation.
- It is much less regular than batch gradient descent: instead of gently decreasing until it reaches the minimum, the cost function will bounce up and down, decreasing only on average.
- When the cost function is very irregular, the randomness helps the algorithm jump out of local minima. However the algorithm never settles at the minimum.
- One solution is to gradually decrease the learning rate: the function that determines the learning rate at each iteration is called the learning schedule.

$$\eta = \frac{c_1}{\text{number of iterations} + c_2}$$

where c_1 and c_2 are constants.

Gradient Descent

- Example: SGD of a linear regression model

```
n_epochs = 50
t0, t1 = 5, 50 # learning schedule hyperparameters

def learning_schedule(t):
    return t0 / (t + t1)

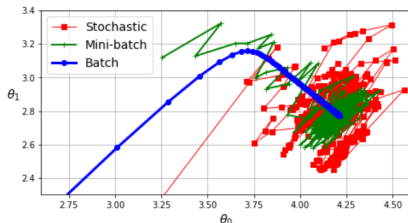
np.random.seed(42)
theta = np.random.randn(2, 1) # random initialization

for epoch in range(n_epochs):
    for iteration in range(m):

        random_index = np.random.randint(m)
        xi = X_b[random_index : random_index + 1]
        yi = y[random_index : random_index + 1]
        gradients = 2 * xi.T @ (xi @ theta - yi) # for SGD, do not divide by m
        eta = learning_schedule(epoch * m + iteration)
        theta = theta - eta * gradients
```

Gradient Descent

- The **Mini-Batch Gradient Descent** computes the gradients on (random) subsamples of the training set, called mini-batches.
- It is a compromise between batch gradient descent and SGD
 - ▶ Convergence is reached faster than batch gradient descent
 - ▶ Compared to the SGD, the algorithm's process in parameter space is less erratic and it will end up walking around a bit closer to the minimum
 - ▶ It is harder to escape from local minima



Géron (2023) Figure 4-11

- Bates, J., & Granger, C. (1969). The combination of forecasts. *Journal of the Operational Research Society*, 20(4), 451–468.
- Clemen, R. (1989). Combining forecasts: A review and annotated bibliography. *International Journal of Forecasting*, 5(4), 559–583.
- Elliott, G., & Timmermann, A. (2004). Optimal forecast combinations under general loss functions and forecast error distributions. *Journal of Econometrics*, 122(1), 47–79.
- Fernandez, C., Ley, E., & Steel, M. (2001). Benchmark priors for bayesian model averaging. *Journal of Econometrics*, 100(2), 381–427.
- Géron, A. (2023). *Hands-on machine learning with scikit-learn, keras & tensorflow* (2nd ed.). O'Reilly Media, Inc.
- Magnus, J., Powell, O., & Prüfer, P. (2010). A comparison of two model averaging techniques with an application to growth empirics. *Journal of Econometrics*, 154(2), 139–153.
- Makridakis, S., & Hibon, M. (2000). The m3-competition: Results, conclusions and implications. *International Journal of Forecasting*, 16(4), 451–476.
- Marcellino, M. (2004). Forecast pooling for european macroeconomic variables. *Oxford Bulletin of Economics and Statistics*, 66(1), 91–112.

- Raftery, A., Madigan, D., & Hoeting, J. (1997). Bayesian model averaging for linear regression models. *Journal of the American Statistical Association*, 92(437), 179–191.
- Stock, J., & Watson, M. (1998). A comparison of linear and nonlinear univariate models for forecasting macroeconomic time series. *NBER Working Paper*, w6607.